Claude for Software Engineers

Use Claude as a force multiplier — not a replacement for your thinking

■ DELEGATE TEDIOUS, NOT THOUGHTFUL

- ■ Boilerplate & scaffolding you already know
- ■ Tests for code *you* wrote
- ■ Type/interface conversion between formats
- ■ Docs and changelogs from your diffs
- ■ Regex & one-off string manipulation

Keep architectural decisions yours.

■ REQUIRE EXPLANATIONS

- ■ Ask Claude to walk through any non-obvious logic
- ■ Don't accept code that surprises you
- ■ Catches hallucinated APIs before they ship
- ■ If you can't explain every line — it shouldn't merge

■■■ TREAT CLAUDE AS A JUNIOR DEV

- ■ You stay decision-maker; Claude executes
- ■ Don't ask *"how to architect?"* — say *"implement X using approach Y"*
- ■ Never merge code you can't explain to a contributor
- ■ Your library's philosophy isn't in Claude's context

■ USE FOR CONTEXT-SWITCHING

- ■ Coming back after 2 weeks away? Paste in files
- ■ Ask: summarize current state, recent decisions
- ■ Ask: how does this module work?
- ■ Low risk, high value — Claude reads code *you* wrote

✍■ WRITE HARD PARTS YOURSELF FIRST

- ■ Draft your own rough implementation, even if ugly
- ■ Then ask Claude to review, improve, or refactor
- ■ Keeps the thinking yours; Claude handles polish
- ■ Outsourcing reasoning = stranger in your own codebase

■ GET REAL FEEDBACK (NOT SYCOPHANCY)

- ■ "What are the strongest arguments **against** this?"
- ■ "Steelman doing this a completely different way."
- ■ "What would have to be true for this to be bad?"
- ■ "What are the non-obvious failure modes?"
- ■ Ask for critique *only* first — no solutions yet
- ■ More specific proposal = better critique

Default behavior leans validating. Ask adversarially.

■ COMMIT DISCIPLINE

- ■ Note in commits when Claude was heavily involved
- ■ Review those PRs with extra scrutiny before merging
- ■ Stays accountable; keeps you reading code carefully

■ CODE REVIEW CLAUDE'S OUTPUT

- ■ Read it like a PR from someone you don't yet trust
- ■ Does it fit existing patterns in my codebase?
- ■ Is this idiomatic? Are errors handled my way?
- ■ Claude doesn't know your style — you do